Trabalho prático 01

João Gabriel Vaz
2023099166
Universidade Federal de Minas Gerais
Computação Natural

17 de outubro de 2026

1 Introdução

A regressão simbólica procura uma expressão matemática que explique pares de entradas e saídas, em vez de ajustar apenas os coeficientes de uma forma funcional previamente fixada. Dado um conjunto com N exemplos e d atributos de entrada, deseja-se encontrar uma função $\widehat{f} : \mathbb{R}^d \to \mathbb{R}$ que aproxime as respostas observadas. Na programação genética (GP), cada expressão é um indivíduo e suas partes são recombinadas ou modificadas ao longo das gerações.

O objetivo é implementar os componentes básicos desse processo e investigar como população, gerações, seleção e probabilidades de variação afetam a qualidade, o custo e a diversidade das soluções. A representação em árvore permite expressões de formatos distintos e facilita a interpretação da solução final, mas cria desafios relacionados à estabilidade numérica, ao crescimento estrutural e à exploração de um espaço de busca extenso.

2 Implementação

2.1 Caminho de execução e configuração

A entrada pública é `python -m gp`, que executa o módulo `src/gp/__main__.py`. O interpretador de argumentos separa três operações: uma execução individual, uma etapa experimental e a análise de resultados já salvos. Em uma execução individual, a sequência é carregar e validar parâmetros, carregar os dados, evoluir a população exclusivamente no treino, avaliar o campeão no teste e persistir os resultados.

O módulo `config.py` concentra os parâmetros em uma dataclass imutável. Isso evita alterações acidentais durante uma execução. Uma nova configuração é criada para cada combinação experimental; os valores fornecidos no terminal sobrescrevem os correspondentes no TOML. A validação rejeita tamanhos ou probabilidades inválidos, funções desconhecidas e limites incompatíveis. Centralizar os parâmetros facilita o estudo de sensibilidade e reduz a possibilidade de comparar execuções com diferenças ocultas, em troca de uma camada adicional de validação e configuração.

2.2 Carregamento e separação dos dados

O módulo `data.py` lê os CSVs numéricos sem cabeçalho. Todas as colunas, exceto a última, formam X; a última forma y. Por exemplo, as linhas $(1, 3, 8)$ e $(2, 4, 10)$ resultam em

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \qquad y = \begin{bmatrix} 8 \\ 10 \end{bmatrix}.$$

O carregamento converte as entradas para arrays de ponto flutuante e verifica dimensões, ausência de valores não finitos e consistência do número de atributos entre treino e teste. Não há normalização dos atributos. Essa decisão preserva as unidades e a interpretação direta das expressões, mas pode tornar operações e constantes menos adequadas quando as variáveis possuem escalas muito diferentes, situação relevante para a base real.

2.3 Representação dos indivíduos e conjunto de primitivas

Cada indivíduo é uma árvore de nós imutáveis definida em `tree.py`. Um nó contém o tipo, um valor e uma tupla de filhos. Os tipos possíveis são função, variável e constante. Em um nó de função, o valor identifica uma primitiva; em uma variável, ele indica a coluna de X; em uma constante, é o próprio número. As aridades são verificadas para impedir estruturas como uma soma com apenas um operando.

A expressão $x_0 + 2x_1$, por exemplo, pode ser construída assim:

```
tree = Node.function(
    "add",
    Node.variable(0),
    Node.function("mul", Node.constant(2), Node.variable(1)),
)
```

Ela possui cinco nós e profundidade dois. A profundidade é medida em arestas, com a raiz no nível zero. Portanto, o limite sete significa até oito níveis e permite até 255 nós em uma árvore inteiramente binária; não significa um limite de sete nós. Essa interpretação é mantida na inicialização e nos operadores.

Os caminhos são tuplas de índices de filhos. Nesse exemplo, $()$ designa a árvore inteira, $(0,)$ designa x_0, $(1,)$ designa $2x_1$ e $(1, 0)$ designa a constante 2. Consultar uma subárvore não a altera; substituí-la produz uma nova árvore e reconstrói os ancestrais necessários. A imutabilidade permite compartilhar os outros trechos e preservar pais e campeões com segurança. Seu custo é a criação de novos nós ancestrais durante as mudanças.

O conjunto padrão de funções, em `primitives.py`, contém soma, subtração, multiplicação, divisão protegida, seno e cosseno. A negação também está implementada e pode ser habilitada. As operações aritméticas fornecem composições polinomiais e racionais; seno e cosseno ampliam a expressividade para comportamentos periódicos. Essa ampliação aumenta o espaço de busca e também permite expressões desnecessariamente complexas.

Os terminais são as d variáveis de entrada e constantes efêmeras amostradas uniformemente em $[-5, 5]$. Uma constante é sorteada na criação do nó e permanece fixa em avaliações posteriores. O intervalo limita a escala inicial dos coeficientes; números externos a ele ainda podem ser construídos por composição, com possível aumento da complexidade.

Todas as primitivas recebem e devolvem números reais, dispensando uma gramática de tipos. O fechamento numérico exige, entretanto, lidar com a divisão próxima de zero. A operação adotada é

$$\mathrm{div}_p(a, b) = \begin{cases} 1, & |b| \le 10^{-12}, \\ a/b, & \text{caso contrário.} \end{cases}$$

A proteção evita divisões indefinidas ou instáveis perto de zero, mas altera a interpretação da divisão nessa região. A expressão textual preserva essa distinção ao utilizar o nome `protected_div`.

2.4 Inicialização ramped half-and-half

O módulo `initialization.py` combina os métodos *full* e *grow*. Full escolhe funções antes do alvo de profundidade e terminais ao alcançá-lo. Grow pode escolher funções ou terminais antes desse limite, produzindo ramos com comprimentos diferentes.

Ramped half-and-half distribui a população entre os alvos de profundidade dois a sete e divide cada grupo entre grow e full. Quando a divisão não é exata, os grupos diferem em no máximo um indivíduo. Os excedentes de grupos ímpares alternam entre os métodos, e a população é embaralhada ao final. Essa distribuição amplia a diversidade inicial de formas e tamanhos. Não garante, porém, unicidade de expressões nem equilíbrio entre profundidades efetivamente observadas: grow pode encerrar antes do alvo, inclusive na raiz.

No grow, a escolha é uniforme entre os nomes de funções, as variáveis e uma opção de constante. Com seis funções e duas entradas, há nove opções, das quais três são terminais. Com oito entradas, há quinze opções, das quais nove são terminais. Logo, manter a mesma regra em bases diferentes também muda a distribuição estrutural inicial. Full pode gerar árvores com números de nós distintos para uma mesma profundidade, pois há funções unárias e binárias.

2.5 Avaliação vetorizada e função de fitness

O módulo `fitness.py` interpreta a árvore recursivamente. A chamada começa na raiz, desce até os terminais e combina os resultados na volta da recursão. Cada nó devolve um vetor com uma saída por amostra: uma variável consulta uma coluna de X, uma constante repete seu valor e uma função combina os vetores dos filhos, posição a posição.

Para o exemplo anterior,

$$x_0 = [1, 2], \quad x_1 = [3, 4], \quad 2x_1 = [6, 8], \quad \widehat{y} = [7, 10].$$

As duas amostras são avaliadas independentemente, mas em operações NumPy vetorizadas. Isso reduz o custo de laços Python por amostra. Ainda existe interpretação recursiva por árvore, e não há cache de avaliações de indivíduos repetidos; a vetorização também exige memória para os vetores intermediários e para a matriz de erros.

A fitness é a raiz do erro quadrático médio:

$$\mathrm{RMSE}(T) = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left(T(x_i) - y_i\right)^2}. \tag{1}$$

Quanto menor o valor, melhor o indivíduo. Para $y = [8, 10]$ e $\widehat{y} = [7, 10]$, o RMSE é $\sqrt{1/2} \simeq 0{,}7071$. Essa medida penaliza mais fortemente grandes resíduos, mas fica sensível a casos extremos. Seu valor bruto não deve ser comparado entre bases com escalas diferentes de resposta.

Para reduzir overflow ao elevar resíduos ao quadrado, o cálculo usa $s = \max_i |\widehat{y}_i - y_i|$ e, quando $s > 0$,

$$\mathrm{RMSE} = s\sqrt{\frac{1}{N}\sum_{i} \left(\frac{|\widehat{y}_i - y_i|}{s}\right)^2}.$$

Quando $s = 0$, o resultado é zero. Uma saída não finita em qualquer caso invalida o indivíduo inteiro. Sua fitness e todos os erros por caso recebem 10^{100}; erros finitos extremos também são limitados a esse teto. Isso mantém os seletores operando com valores finitos e impede que uma expressão parcialmente inválida seja favorecida em alguns casos, mas deixa de distinguir magnitudes acima do teto.

Além do RMSE por indivíduo, são armazenados erros absolutos $e_{it} = |T_i(x_t) - y_t|$ em uma matriz de dimensão população por casos, utilizada pela seleção ε-lexicase.

2.6 Seleção de pais

2.6.1 Roleta

O módulo `selection.py` transforma o RMSE f_i em um peso $w_i = 1/(1+f_i)$ e seleciona o indivíduo com probabilidade

$$p_i = \frac{w_i}{\sum_j w_j}.$$

A divisão intermediária pelo maior peso preserva as proporções e melhora a escala numérica antes da normalização pela soma. Para RMSEs $[1, 3, 7]$, os pesos iniciais são $[1/2, 1/4, 1/8]$ e as probabilidades são $[4/7, 2/7, 1/7]$. A regra aceita erro zero, mas sua pressão seletiva depende das diferenças numéricas e da escala da fitness; o melhor indivíduo recebe maior chance, sem seleção garantida.

2.6.2 Torneio

O torneio sorteia participantes com reposição e escolhe o menor RMSE entre eles, com desempate aleatório. O tamanho padrão é dois. Esse mecanismo é simples e depende da ordenação das fitness, sem exigir probabilidades proporcionais às suas magnitudes. Torneios maiores elevam a pressão de seleção e podem reduzir a diversidade. O mesmo índice pode participar mais de uma vez por causa da reposição.

2.6.3 ε-lexicase

Em cada seleção, os casos de treino são embaralhados. Inicialmente, todos os indivíduos são candidatos. Para cada caso t, calcula-se entre os candidatos atuais C:

$$m_t = \text{mediana}_{i \in C}(e_{it}), \tag{2}$$

$$\varepsilon_t = \text{mediana}_{i \in C}\, |e_{it} - m_t|, \tag{3}$$

$$C' = \{i \in C : e_{it} \leq \min_{j \in C} e_{jt} + \varepsilon_t\}. \tag{4}$$

Se apenas um candidato sobreviver, ele é escolhido. Caso se esgotem os casos, sorteia-se entre os restantes. O MAD é recalculado depois de cada filtragem; a implementação usa $e_{it} - e_t^* \leq \varepsilon_t$ para evitar overflow na soma.

Adotou-se desigualdade não estrita para preservar os melhores quando MAD é zero. Por exemplo, erros $[1, 1, 4]$ possuem mediana 1 e MAD 0: o filtro $e < 1$ excluiria todos, enquanto $e \leq 1$ preserva os dois melhores. Essa escolha resolve o caso limite da expressão estrita apresentada no enunciado [1].

O mecanismo pode selecionar especialistas em casos diferentes sem reduzir toda a decisão a um único RMSE agregado. Em contrapartida, requer manter a matriz de erros e executar filtragens e medianas repetidas. Essa diferença de custo precisa ser confrontada com o ganho obtido, em vez de presumir superioridade universal.

2.7 Operadores, probabilidades e substituição geracional

O módulo `operators.py` escolhe posições uniformemente entre todos os nós, incluindo a raiz. O crossover troca uma subárvore de cada pai e gera dois candidatos. Cada filho excessivo em profundidade é substituído pelo pai correspondente; o outro continua disponível. Não há repetição automática do sorteio até obter uma troca válida. A regra é simples e preserva o limite, mas pode desperdiçar propostas e introduzir cópias dos pais.

Por exemplo, sejam $T_1 = x_0 + 2x_1$ e $T_2 = x_1 - 1$. Trocar a subárvore $2x_1$ do primeiro pelo termo x_1 do segundo produz $x_0 + x_1$ e $2x_1 - 1$. Os pais permanecem intactos. Trocas de raízes ou trechos idênticos podem produzir apenas permutações ou cópias, contabilizadas como resultados válidos.

A mutação substitui uma subárvore por outra gerada com grow. Se o ponto escolhido está no nível h, a nova árvore recebe profundidade máxima $7-h$. Assim, o limite é respeitado por construção. Substituir a constante 2 de $x_0 + 2x_1$ por $\sin(x_0)$ produz $x_0 + \sin(x_0)x_1$, com profundidade três. Grow permite tanto expansão quanto redução; usar full com todo o orçamento restante faria o ramo modificado atingir o limite em toda mutação. Ramped half-and-half organiza um conjunto de indivíduos, enquanto essa operação precisa de uma única subárvore.

Em `evolution.py`, crossover, mutação e reprodução são alternativas exclusivas por evento:

$$P(\text{cross}) = p_c, \quad P(\text{mut}) = p_m, \quad P(\text{rep}) = 1 - p_c - p_m, \qquad p_c + p_m \leq 1.$$

Para $(p_c, p_m) = (0{,}6, 0{,}05)$, a reprodução tem probabilidade 0,35. Como crossover gera dois filhos e os outros eventos geram um, a fração de descendentes de crossover não é p_c. Desconsiderando elites e a última vaga, sua fração esperada é aproximadamente $2p_c/(1 + p_c)$. Se resta uma única vaga, somente o primeiro filho do último crossover é inserido.

Cada geração começa com a cópia dos elites. Os pais são selecionados da população anterior até que a nova população esteja completa. Elitismo um preserva a melhor fitness por geração, mas também mantém pressão para reutilizar a solução atual. Independentemente do elitismo, um arquivo em memória preserva o campeão histórico de treino. Assim, sem elitismo, a população poderia piorar mesmo que o campeão histórico continuasse salvo.

Não há penalidade de parcimônia nem simplificação algébrica. A restrição de profundidade contém o crescimento, mas não elimina introns ou redundância. Evitar simplificações preserva a estrutura realmente evoluída e o significado da divisão protegida, ao custo de fórmulas potencialmente longas.

2.8 Métricas, persistência e verificação

O módulo `metrics.py` registra, desde a geração zero, melhor, pior e média da fitness, número de repetidos, tamanho e profundidade médios e máximos, inválidos e efeitos dos operadores. Repetidos são contados como população menos assinaturas estruturais distintas: $x_0 + x_1$ e $x_1 + x_0$ não são considerados iguais por essa métrica.

Cada filho de crossover é comparado ao RMSE médio dos dois pais; mutação e reprodução são comparadas ao único pai. Há três resultados: melhor, igual ou pior. Um filho com RMSE 9 é melhor que a média de pais com RMSEs 8 e 12, embora seja pior que o melhor deles. Além disso, o retorno a um pai após rejeição de crossover pode ser classificado como melhor que a média dos pais. Portanto, a contagem mede o critério operacional registrado, e não prova ganho causal de toda nova estrutura.

O módulo `experiments.py` salva os artefatos da Tabela 1. Cada execução recebe um diretório único. Um hash dos parâmetros sem a semente identifica a configuração; a semente identifica a repetição. O resumo é escrito por último e marca uma execução concluída.

Tabela 1: Arquivos persistidos em cada execução.

Arquivo	Conteúdo
`config.json`	Parâmetros efetivamente utilizados e semente.
`best_tree.json`	Estrutura reconstruível do campeão histórico.
`history.csv`	Estatísticas de cada geração, de 0 até G.
`operators.csv`	Eventos, índices dos pais, referências de fitness, filhos e rejeições.
`summary.json`	RMSE de treino/teste, expressão, tamanho, profundidade, duração e versões do ambiente.

A duração registrada inclui evolução e avaliação final no teste, mas exclui o carregamento anterior dos CSVs e a persistência posterior. A suíte de testes verifica representação, avaliação, seleção, limites, reprodução por semente, métricas, leitura de dados, persistência e interface de terminal.

3 Experimentos e análise dos resultados

3.1 Protocolo, fontes e unidade de repetição

O estudo disponível é sequencial: primeiro varia a população; em seguida, o número de gerações; depois seleção, crossover, etc. Cada grupo contém exatamente as sementes de 0 a 29. As demais escolhas fixadas em cada etapa são descritas na Tabela 2. O limite de profundidade é sete e o elitismo é um em todos os grupos.

Tabela 2: Condições efetivamente registradas nos estudos de `synth1`.

Etapa	População	Gerações	Seleção	p_c	p_m	Execuções
População	50/100/500	100	Torneio (2)	0,9	0,05	90
Gerações	500	50/100/500	Torneio (2)	0,9	0,05	90
Seleção	500	500	Três métodos	0,9	0,05	90
Crossover	500	500	ε-lexicase	0,9/0,6	0,05	60
Mutação	500	500	ε-lexicase	0,6	0,05/0,3	60

Os resultados são apresentados como média e desvio-padrão amostral entre as 30 sementes. Medianas e diferenças pareadas complementam a descrição quando a distribuição é assimétrica. As comparações não constituem testes de significância. Nos estudos de gerações, as trajetórias por semente têm prefixos idênticos, permitindo observar o efeito de continuar a mesma evolução. Em mudanças de população ou de seleção, a mesma semente não garante a mesma sequência efetiva de eventos, pois o consumo do gerador aleatório também muda.

O conjunto de teste é utilizado somente depois da evolução. As justificativas de escolha apresentadas a seguir usam os indicadores de treino; os valores de teste são diagnóstico de generalização.

3.2 Interface de execução e significado dos parâmetros

Os comandos pressupõem execução na raiz do projeto e Python 3.11 ou superior, conforme a configuração do pacote; a leitura de TOML utiliza `tomllib`, disponível a partir dessa versão. A preparação em uma nova instalação pode ser feita por:

```
cd /caminho/para/tp1-compnat
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
python -m pytest -q
```

O primeiro comando permite conferir a versão antes de criar o ambiente; se ela for anterior a 3.11, deve-se selecionar um interpretador compatível. O ambiente virtual isola as dependências. A instalação editável disponibiliza o pacote `gp`; o grupo `dev` inclui pytest. O último comando executa os testes, sem conduzir a bateria de experimentos.

3.2.1 Uma execução: `run`

Tabela 3: Argumentos do subcomando `run`.

Argumento	Padrão	Efeito
`-dataset`	Obrigatório	Escolhe `synth1`, `synth2`, `concrete` ou o alias `concrete1`.

Argumento	Padrão	Efeito
`-config`	`configs/baseline.toml`	Carrega os parâmetros GP de um arquivo TOML.
`-data-dir`	`bases-TP1/datasets`	Define a raiz com as subpastas das bases e seus CSVs.
`-output-dir`	`results/runs`	Define o diretório que receberá uma nova subpasta de execução.
`-seed`	Valor do TOML	Define a semente de uma execução; repetir a mesma configuração e ambiente reproduz a evolução.
`-population-size`	Valor do TOML	Sobrescreve a quantidade de indivíduos mantida em cada geração.
`-generations`	Valor do TOML	Sobrescreve o número de gerações após a população inicial. O histórico possui $G + 1$ linhas.
`-max-depth`	Valor do TOML	Sobrescreve o limite de profundidade em arestas.
`-min-initial-depth`	Valor do TOML	Sobrescreve o menor alvo dos grupos ramped; precisa ser menor ou igual ao limite.
`-selection`	Valor do TOML	Escolhe `tournament`, `roulette` ou `epsilon_lexicase`.
`-h`, `-help`	–	Mostra a ajuda e encerra sem executar a evolução.

O exemplo a seguir demonstra todos os controles específicos de uma execução:

```
python -m gp run \
  --dataset synth1 \
  --config "configs/baseline copy.toml" \
  --data-dir bases-TP1/datasets \
  --output-dir results/exemplo-individual \
  --seed 7 \
  --population-size 50 \
  --generations 100 \
  --max-depth 7 \
  --min-initial-depth 2 \
  --selection tournament
```

Ele realiza uma execução em `synth1`, com 50 indivíduos, 100 gerações, torneio e semente 7. Os argumentos substituem os valores correspondentes do arquivo; p_c, p_m, elitismo, funções e constantes continuam vindo do TOML. Assim, com o arquivo atual, esse exemplo usa $p_c = 0{,}6$ e $p_m = 0{,}05$. É uma demonstração de interface, não a reconstrução automática da primeira etapa histórica. Para uma configuração inteiramente definida pelo arquivo:

```
python -m gp run --dataset concrete \
  --config "configs/baseline copy.toml" \
  --output-dir results/exemplo-concrete
```

Esse segundo comando executaria uma única semente na base real com o estado atual do arquivo.

3.2.2 Uma bateria: `experiment`

Tabela 4: Argumentos do subcomando `experiment`.

Argumento	Padrão	Efeito
`-stage`	Obrigatório	Seleciona uma etapa existente no plano.
`-plan`	`configs/experiments.toml`	Escolhe o arquivo que define bases, sementes, variantes e valores fixos das etapas.

Argumento	Padrão	Efeito
`-baseline`	Arquivo indicado no plano	Escolhe a configuração de partida. O parâmetro variado pela etapa substitui seu valor no baseline.
`-seeds`	0 a 29 no plano	Substitui a lista de sementes por um ou mais inteiros; não aceita repetições na mesma lista.
`-datasets`	Lista do plano/etapa	Substitui a lista de bases, permitindo estudar uma de cada vez.
`-data-dir`	`bases-TP1/datasets`	Altera a raiz dos CSVs.
`-output-dir`	`results/runs`	Escolhe o diretório comum das subpastas produzidas pela bateria.
`-dry-run`	Desativado	Expande e valida o plano, exibindo contagens e identificadores, sem evoluir populações.
`-h`, `-help`	–	Mostra a ajuda desse subcomando.

Por exemplo:

```
python -m gp experiment \
  --plan configs/experiments.toml \
  --stage population \
  --baseline configs/baseline.toml \
  --datasets synth1 \
  --seeds 0 1 \
  --data-dir bases-TP1/datasets \
  --output-dir results/exemplo-populacao \
  --dry-run
```

O programa mostraria seis execuções: três populações, 50, 100 e 500, para duas sementes. Retirar `--dry-run` executaria essa bateria. Retirar também `--seeds 0 1` restauraria as 30 sementes do plano, totalizando 90 execuções em uma base. Usar `--datasets synth1 concrete` incluiria as duas bases; com as 30 sementes, seriam 180 execuções nessa etapa.

As etapas disponíveis são `population`, `generations`, `selection`, `crossover`, `mutation`, `operators`, `elitism` e `transfer`. A etapa `operators` compara os pares $(0{,}9, 0{,}05)$ e $(0{,}6, 0{,}3)$ simultaneamente; não isola o efeito de cada probabilidade.

Os parâmetros p_c, p_m, tamanho do torneio, quantidade de elites, conjunto de funções e intervalo de constantes são alterados no TOML, pois não possuem flags próprias. O estado do arquivo deve ser atualizado ou copiado conscientemente entre etapas; o programa não reescreve parâmetros automaticamente.

3.2.3 Tabelas e gráficos: `analyze`

O comando sem argumentos:

```
python -m gp analyze
```

lê subpastas de `results/runs`, grava `results/summaries/summary.csv` e cria PNGs em `results/figures`. Para analisar uma bateria salva em outro lugar:

```
python -m gp analyze \
  --runs-dir results/estudo-populacao-synth1 \
  --summaries-dir results/tabelas-populacao-synth1 \
  --figures-dir results/figuras-populacao-synth1
```

`--runs-dir` define a pasta que contém diretamente os diretórios das execuções; a busca não agrega recursivamente todas as subpastas arbitrárias de `results`. `--summaries-dir` altera o destino da tabela e `--figures-dir` altera o destino das imagens. São os três parâmetros específicos desse

subcomando, além de `-help`.

O módulo `reporting.py` agrupa por base, etapa e configuração, calcula média e desvio-padrão amostral e desenha três painéis com Matplotlib: convergência do melhor RMSE de treino, número médio de nós e RMSE final de treino/teste. As faixas das curvas são média mais ou menos um desvio-padrão, com limite inferior truncado em zero, isto é, $\max(0, \overline{x} - s)$. Os arquivos usam o padrão `base-etapa.png`.

Uma única repetição deixa o desvio-padrão em branco. Sementes repetidas na mesma base, etapa e configuração são rejeitadas na agregação; novas tentativas devem usar diretórios separados.

Para consultar a interface instalada:

```
python -m gp --help
python -m gp run --help
python -m gp experiment --help
python -m gp analyze --help
```

3.3 Tamanho da população

Tabela 5: Sensibilidade à população em synth1, com $G = 100$, torneio, $p_c = 0{,}9$ e $p_m = 0{,}05$. Valores: média ± desvio-padrão amostral de 30 sementes.

Variante	RMSE de treino	RMSE de teste	Tempo (s)
$P = 50$	$7{,}0687 \pm 4{,}4395$	$16{,}4698 \pm 23{,}1013$	$1{,}45 \pm 0{,}29$
$P = 100$	$4{,}3138 \pm 2{,}5741$	$56{,}0186 \pm 162{,}4864$	$3{,}03 \pm 0{,}56$
$P = 500$	$2{,}0746 \pm 1{,}2972$	$3{,}0291 \pm 2{,}3808$	$16{,}31 \pm 2{,}09$

Com 100 gerações e os demais parâmetros fixos, aumentar a população de 50 para 500 reduziu o RMSE médio de treino em 70,65%. Em relação à população 100, a redução foi 51,91%, com erro menor em 24 das 30 sementes. Isso sustenta a escolha de 500 para as etapas seguintes quando a prioridade é qualidade. O custo foi aproximadamente 5,39 vezes o de 100 indivíduos e 11,28 vezes o de 50 nessa etapa. Populações menores continuam justificáveis para testes rápidos ou restrições de tempo.

A generalização não melhorou monotonicamente com esse parâmetro. Para população 100, o RMSE médio de teste foi 56,0186, mas a mediana foi 6,8371 e o máximo 748,9166. Portanto, poucos casos extremos dominaram a média. Não se deve concluir que toda execução com 100 indivíduos generaliza pior do que toda execução com 50. A população 500 apresentou menor média de treino e uma distribuição de teste menos extrema no conjunto observado.

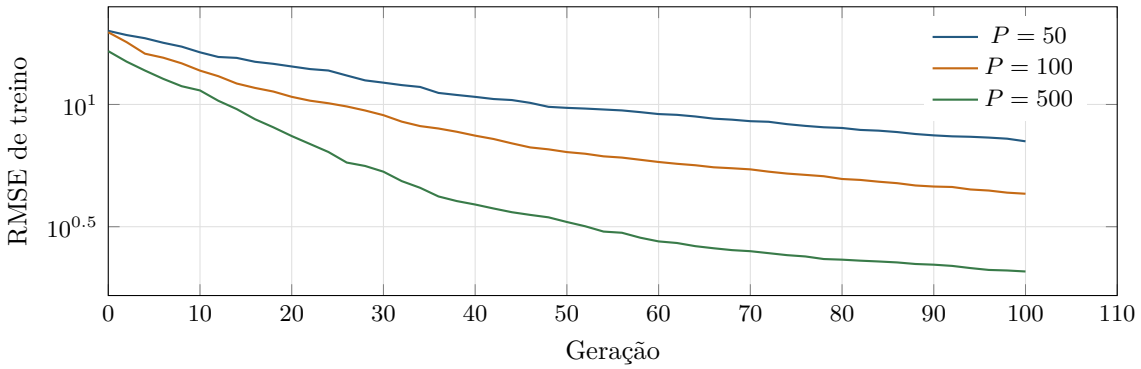


Figura 1: Melhor RMSE de treino por geração, em média entre 30 sementes, para as três populações. O eixo vertical é logarítmico; as curvas não representam intervalos de confiança.

3.4 Número de gerações e orçamento computacional

Tabela 6: Sensibilidade ao número de gerações em synth1, com $P = 500$, torneio, $p_c = 0{,}9$ e $p_m = 0{,}05$. Valores: média ± desvio-padrão amostral de 30 sementes.

Variante	RMSE de treino	RMSE de teste	Tempo (s)
$G = 50$	$3{,}3060 \pm 1{,}5962$	$4{,}7186 \pm 3{,}2102$	$7{,}84 \pm 1{,}15$
$G = 100$	$2{,}0746 \pm 1{,}2972$	$3{,}0291 \pm 2{,}3808$	$16{,}17 \pm 2{,}08$
$G = 500$	$1{,}0080 \pm 0{,}7166$	$1{,}6736 \pm 1{,}8277$	$95{,}64 \pm 13{,}87$

Com 500 indivíduos, continuar de 100 para 500 gerações reduziu o RMSE médio de treino de 2,0746 para 1,0080, uma queda de 51,41%. Todas as 30 sementes melhoraram no treino; no teste, 27 melhoraram e três pioraram. O tempo médio passou de 16,172 para 95,643 segundos, aproximadamente 5,91 vezes maior. A escolha de 500 gerações privilegia a qualidade observada, com retorno decrescente em relação ao custo. Ela não demonstra que 500 seja um ótimo global, nem que continuar indefinidamente seja vantajoso.

3.5 Mecanismos de seleção

Tabela 7: Comparação de seleção em synth1, com $P = G = 500$, $p_c = 0{,}9$ e $p_m = 0{,}05$. Valores: média ± desvio-padrão amostral de 30 sementes.

Variante	RMSE de treino	RMSE de teste	Tempo (s)
Torneio	$1{,}0080 \pm 0{,}7166$	$1{,}6736 \pm 1{,}8277$	$101{,}31 \pm 15{,}70$
Roleta	$1{,}2335 \pm 0{,}7244$	$7{,}1237 \pm 24{,}9461$	$95{,}43 \pm 14{,}78$
ε-lexicase	$0{,}2393 \pm 0{,}5076$	$1{,}3568 \pm 5{,}2974$	$329{,}10 \pm 41{,}97$

Com população e gerações fixadas em 500, ε-lexicase reduziu o RMSE médio de treino em 76,26% em relação ao torneio e em 80,60% em relação à roleta. Em cada comparação, obteve menor erro em 27 das 30 sementes. O tempo médio foi 329,101 segundos, contra 101,307 do torneio e 95,432 da roleta: aproximadamente 3,25 e 3,45 vezes maior, respectivamente. A vantagem de qualidade observada sustenta sua adoção no estudo sucessivo, ao custo de um seletor mais oneroso.

As medianas de treino foram 0,6799 no torneio, 1,2877 na roleta e 0,0454 em ε-lexicase. No teste, as medianas foram 0,7804, 1,9618 e 0,0508. Apesar do bom resultado típico da lexicase, seu desvio de teste foi elevado e o pior caso chegou a 29,0085. Logo, o ganho médio não elimina a ocorrência de expressões frágeis fora dos pontos de treino. A grande diferença entre média e mediana recomenda avaliar ambas e examinar resíduos individuais.

A redução de erro é compatível com o benefício de filtrar por casos em vez de usar somente o RMSE agregado, mas os arquivos não demonstram que a diversidade estrutural foi necessariamente maior. De fato, as contagens de repetidos finais indicam outro comportamento, discutido adiante.

3.6 Probabilidade de crossover

Tabela 8: Sensibilidade ao crossover em synth1, com $P = G = 500$, ε-lexicase e $p_m = 0{,}05$. Valores: média ± desvio-padrão amostral de 30 sementes.

Variante	RMSE de treino	RMSE de teste	Tempo (s)
$p_c = 0{,}9$	$0{,}2393 \pm 0{,}5076$	$1{,}3568 \pm 5{,}2974$	$322{,}98 \pm 40{,}52$
$p_c = 0{,}6$	$0{,}2367 \pm 0{,}4179$	$1{,}2215 \pm 3{,}9099$	$329{,}21 \pm 36{,}54$

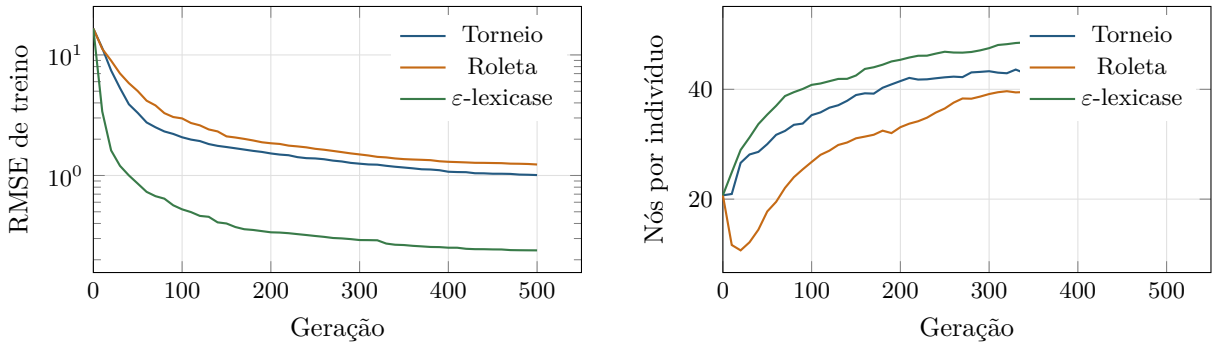


Figura 2: Seleção em synth1: melhor RMSE de treino (à esquerda, eixo vertical logarítmico) e tamanho médio dos indivíduos (à direita). Cada ponto representa a média de 30 execuções; as faixas de dispersão não são exibidas.

Reduzir p_c de 0,9 para 0,6, mantendo $p_m = 0{,}05$, reduziu a média de treino de 0,239284 para 0,236736: apenas 1,06%. A diferença pareada $(0{,}6 - 0{,}9)$ foi $-0{,}00255 \pm 0{,}50007$, com vantagem de 0,6 em 18 sementes e de 0,9 em 12. A mediana de treino favoreceu ligeiramente 0,9: 0,04542 contra 0,04836. Portanto, a evidência não sustenta uma superioridade ampla de 0,6.

A configuração 0,6 foi mantida por apresentar a menor média de treino no critério adotado e árvores campeãs menores em média: 46,6 contra 53,5 nós. O tempo médio foi semelhante e ligeiramente maior com 0,6, mostrando que menos crossovers não implica automaticamente execução mais rápida. Ao diminuir p_c nessa comparação, a reprodução direta aumentou de 0,05 para 0,35. Assim, o contraste também altera a preservação de cópias, com reflexos na duplicação da população.

3.7 Probabilidade de mutação e escolha final

Tabela 9: Sensibilidade à mutação em synth1, com $P = G = 500$, ε-lexicase e $p_c = 0{,}6$. Valores: média ± desvio-padrão amostral de 30 sementes.

Variante	RMSE de treino	RMSE de teste	Tempo (s)
$p_m = 0{,}05$	$0{,}2367 \pm 0{,}4179$	$1{,}2215 \pm 3{,}9099$	$332{,}98 \pm 37{,}33$
$p_m = 0{,}3$	$0{,}3376 \pm 0{,}6602$	$1{,}1056 \pm 2{,}7207$	$324{,}33 \pm 49{,}42$

Com $p_c = 0{,}6$, manter $p_m = 0{,}05$ produziu RMSE médio de treino 29,87% menor que usar 0,3. Entretanto, a distribuição muda essa interpretação: a mutação 0,3 obteve menor erro em 20 das 30 sementes e mediana 0,00979, contra 0,04836 com 0,05. A diferença pareada $(0{,}3 - 0{,}05)$ foi $0{,}10085 \pm 0{,}82266$; poucas execuções ruins elevaram a média de 0,3.

No teste, 0,3 apresentou média 1,10555 e mediana 0,04082, menores que 1,22148 e 0,06506 de 0,05. A menor taxa não domina a maior: a escolha depende de privilegiar a média de treino, o comportamento típico ou a frequência de êxito. Mais repetições e uma análise previamente definida de robustez ajudariam a resolver o conflito. Não se afirma significância estatística ou universalidade da escolha.

Aumentar p_m reduz a reprodução direta de 0,35 para 0,10. Isso amplia as substituições e reduz a preservação clonal, podendo recuperar diversidade ou destruir bons trechos. Os campeões com 0,3 tiveram 53,77 nós em média, contra 46,60 com 0,05. A Figura 3 relaciona a evolução do erro médio e da fração de duplicados nessa comparação.

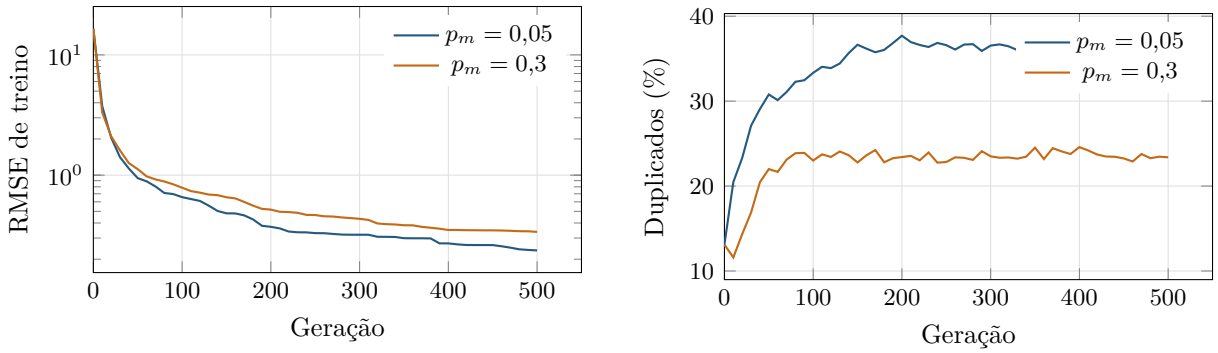


Figura 3: Efeito da mutação com $p_c = 0{,}6$ e ε-lexicase. À esquerda: média do melhor RMSE de treino, com eixo vertical logarítmico. À direita: fração média de duplicados estruturais. Médias de 30 sementes, sem faixas de dispersão.

3.8 Contribuição dos operadores e comparação dos pares sugeridos

As Tabelas 10 e 11 apresentam contagens e proporções ao longo de todas as gerações, com os resultados das 30 sementes de cada grupo. O denominador das taxas é o total de filhos *inseridos daquele operador*; elites e filhos descartados por falta de vaga não entram nesse denominador. Melhor/igual/pior seguem a referência de fitness descrita na Seção 2.

Tabela 10: Contagens de filhos inseridos, agrupadas nas 30 sementes. Cada rótulo indica seleção e p_c/p_m; ε designa ε-lexicase. $+$, $=$ e $-$ indicam fitness melhor, igual e pior que a referência parental.

Configuração	Cross. +	Cross. =	Cross. −	Mut. +	Mut. =	Mut. −
ε 0,9/0,05	1 466 515	182 754	5 441 868	12 575	10 971	173 635
ε 0,6/0,05	1 142 703	202 897	4 266 829	17 042	11 903	206 129
ε 0,6/0,3	1 161 739	224 983	4 225 972	99 720	89 267	1 215 034

Tabela 11: Taxas percentuais de resultados dos operadores. Os totais são agrupados entre sementes antes da divisão; cada trio soma aproximadamente 100%. Rótulos: seleção e p_c/p_m.

Configuração	Cross. +	Cross. =	Cross. −	Mut. +	Mut. =	Mut. −
Torneio 0,9/0,05	31,41	1,36	67,23	14,85	11,30	73,85
Roleta 0,9/0,05	29,84	5,57	64,59	16,31	11,82	71,87
ε 0,9/0,05	20,68	2,58	76,74	6,38	5,56	88,06
ε 0,6/0,05	20,36	3,62	76,02	7,25	5,06	87,69
ε 0,6/0,3	20,70	4,01	75,29	7,10	6,36	86,54

Nos dois pares $(0{,}9, 0{,}05)$ e $(0{,}6, 0{,}3)$, a maioria dos filhos foi pior que sua referência imediata. Isso não contradiz a melhora do campeão: a busca avalia muitas propostas e conserva as melhores por seleção e elitismo. Também não permite concluir que os operadores sejam dispensáveis, pois a comparação com reprodução pura não foi realizada.

Em ε-lexicase, as taxas de melhora de crossover ficaram próximas de 20% e as de mutação próximas de 6–7%. No torneio e na roleta, essas frações foram maiores, mas as soluções finais tiveram erros maiores. Uma referência parental já muito boa torna mais difícil produzir uma melhora imediata. A taxa local de sucesso não pode ser usada sozinha para ordenar os métodos de seleção.

Os contadores incluem o retorno ao pai quando um crossover é rejeitado por profundidade. Esse retorno pode melhorar a média dos pais sem introduzir uma nova estrutura. A taxa de rejeição por proposta foi 23,88% para o par lexicase $(0{,}9, 0{,}05)$, 22,53% para $(0{,}6, 0{,}05)$ e 24,49% para $(0{,}6, 0{,}3)$.

Portanto, a análise distingue sucesso operacional, rejeição estrutural e melhoria efetiva do campeão.

3.9 Convergência, diversidade e bloating

Tabela 12: Estrutura e duplicação na geração 500: médias ± desvios entre 30 sementes. Nós e profundidade são primeiro promediados dentro de cada população. Rejeição considera propostas de filhos de crossover ao longo da execução; duplicados e rejeição estão em porcentagem. Rótulos: seleção e p_c/p_m.

Configuração	Nós	Profundidade	Duplicados	Rejeição
Torneio 0,9/0,05	$47{,}41 \pm 10{,}23$	$6{,}72 \pm 0{,}17$	$13{,}91 \pm 4{,}17$	$21{,}76 \pm 2{,}39$
Roleta 0,9/0,05	$43{,}77 \pm 12{,}70$	$6{,}67 \pm 0{,}26$	$15{,}47 \pm 7{,}34$	$19{,}87 \pm 2{,}67$
ε 0,9/0,05	$51{,}05 \pm 12{,}20$	$6{,}85 \pm 0{,}11$	$24{,}34 \pm 5{,}88$	$23{,}88 \pm 2{,}35$
ε 0,6/0,05	$45{,}00 \pm 10{,}91$	$6{,}83 \pm 0{,}15$	$36{,}77 \pm 12{,}47$	$22{,}53 \pm 2{,}48$
ε 0,6/0,3	$52{,}53 \pm 10{,}40$	$6{,}89 \pm 0{,}06$	$23{,}39 \pm 8{,}22$	$24{,}49 \pm 2{,}28$

Para população 500, o tamanho inicial médio era 20,70 nós. Nos estudos de 500 gerações, as populações finais ficaram entre aproximadamente 43,77 e 52,53 nós em média. As profundidades médias finais ficaram entre 6,67 e 6,89, próximas do teto sete. Há crescimento estrutural e pressão sobre o limite, compatíveis com bloating. O crescimento, isoladamente, não prova que os nós adicionais sejam inúteis; seria necessário analisar semântica ou comparar um mecanismo de parcimônia.

A duplicação final aumentou de 24,34% com $p_c = 0{,}9$ para 36,77% com $p_c = 0{,}6$, mantendo $p_m = 0{,}05$. Isso é compatível com o aumento da reprodução direta. Elevar a mutação para 0,3 reduziu a duplicação para 23,39%. Ainda assim, assinaturas diferentes não garantem comportamentos numéricos diferentes, e assinaturas iguais não medem diretamente a diversidade de especialistas por caso.

Nas últimas 50 gerações, houve melhora acima da tolerância numérica em 12/30 execuções de torneio, 13/30 de roleta, 11/30 de lexicase com $(0{,}9, 0{,}05)$, 13/30 com $(0{,}6, 0{,}05)$ e 14/30 com $(0{,}6, 0{,}3)$. Os ganhos absolutos médios foram, respectivamente, 0,0260; 0,0353; 0,00441; 0,02536; e 0,01006. A maior parte da melhora global ocorreu antes, mas não há convergência universal nas 500 gerações.

Não houve indivíduos marcados como inválidos nas trajetórias observadas. Após remover as cópias entre etapas, foram examinados 40.309.500 registros de indivíduos por geração; esses registros não são indivíduos únicos nem observações independentes. O maior indivíduo observado possuía 186 nós, com profundidade máxima sete. A ausência de inválidos não impede picos finitos: na configuração final, a média entre sementes do pior RMSE da população final foi cerca de 19,5 milhões, com mediana 642,5. A média populacional de RMSE foi cerca de 41.176, com mediana 10,99. Consequentemente, pior fitness e média populacional podem ser dominadas por poucas expressões extremas, mesmo quando o campeão é preciso.

Tabela 13: Melhor, média e pior fitness populacional de treino para a configuração escolhida ($P = G = 500$, ε-lexicase, $p_c = 0{,}6$, $p_m = 0{,}05$). Cada célula resume a respectiva estatística de população entre 30 sementes, com média ± desvio amostral. A escala dos extremos evidencia assimetria.

Geração	Melhor RMSE	RMSE médio	Pior RMSE
0	$16{,}521 \pm 2{,}683$	$(1{,}00 \pm 3{,}94)\,10^4$	$(0{,}44 \pm 1{,}95)\,10^7$
100	$0{,}657 \pm 0{,}841$	$173{,}574 \pm 676{,}495$	$(0{,}39 \pm 1{,}41)\,10^5$
500	$0{,}237 \pm 0{,}418$	$(0{,}41 \pm 2{,}25)\,10^5$	$(0{,}20 \pm 1{,}07)\,10^8$

4 Conclusões

A implementação reúne os componentes de GP em árvores, com parâmetros externos, avaliação numericamente protegida e registros suficientes para acompanhar desempenho, diversidade e crescimento. A separação entre evolução no treino, teste final e agregação por sementes torna possível examinar tanto qualidade quanto variabilidade.

Referências

[1] PAPPA, Gisele L. *Trabalho Prático 1: Programação Genética*. Universidade Federal de Minas Gerais, disciplina de Computação Natural, 2026.

[2] PAPPA, Gisele L. *Programação Genética*. Universidade Federal de Minas Gerais, disciplina de Computação Natural, [s.d.]. Slides disponibilizados como material suplementar.